

TRANSFORMER

From Scratch → Architecture → Mathematics → LLMs → Fine-Tuning → Interview

Purpose: A systematic, teaching-friendly and interview-oriented guide to understanding Transformers and how they connect to modern LLMs such as GPT, Llama and Qwen.

1. What is a Transformer?

Definition: A Transformer is a neural-network architecture designed to model relationships between elements in sequential or structured data. Its central idea is **attention**: each token can determine which other tokens are relevant to it.

```
Text
↓
Tokenizer
↓
Token IDs
↓
Embeddings + positional information
↓
Transformer blocks
↓
Logits
↓
Probabilities
↓
Next token
```

- Transformer is an architecture, not one particular model.
- LLMs such as GPT, Llama and Qwen are commonly built from Transformer architectures.
- A Transformer can also be used outside LLMs: BERT-style NLP models, Vision Transformers and multimodal systems.

Key point: Transformer $\neq$ LLM. An LLM is a large language model; many modern LLMs use the Transformer architecture.

2. Why were Transformers introduced?

Before Transformers, sequence problems were often handled with RNNs and LSTMs. They process information sequentially, which can make long-range dependencies difficult and limits parallelism during training.

RNN-style idea:
token 1 $\rightarrow$ token 2 $\rightarrow$ token 3 $\rightarrow$ token 4 $\rightarrow$...

Transformer idea:
tokens can interact through attention
token 1 $\leftrightarrow$ token 2 $\leftrightarrow$ token 3 $\leftrightarrow$ token 4

- Transformers allow much more parallel computation during training.
- Self-attention provides direct paths between distant tokens.
- They scale effectively to very large models and datasets.

Key point: The key innovation to remember is not simply 'it is fast'; it is the attention-based way of computing contextual representations.

3. Where are Transformers used?

Transformers are useful whenever relationships between elements matter.

- **Language:** classification, translation, summarization, question answering, generation and chat.
- **LLMs:** GPT, Llama, Qwen, Mistral and many other language models.
- **Vision:** Vision Transformer (ViT) and related architectures.
- **Speech/audio:** speech recognition, representation learning and generation.

- **Multimodal:** systems combining text, images, audio or other modalities.

4. Transformer vs LLM

Concept	Meaning	Example
Transformer	Neural-network architecture	Encoder, decoder, encoder-decoder
LLM	Large language model	Llama, GPT, Qwen
Model	A trained instance of an architecture	A particular Llama checkpoint
Fine-tuning	Adapting a pretrained model to a task/domain	SFT, LoRA, QLoRA
Library/toolkit	Software used to implement/optimize the workflow	Transformers, PEFT, Unsloth

Architecture → Transformer
 Model → Llama
 Fine-tuning method → QLoRA
 Optimization toolkit → Unsloth

5. High-level Transformer pipeline

```

Raw text
↓
Tokenizer
↓
Token IDs
↓
Token embeddings
+
Positional information
↓
Transformer Block 1
↓
Transformer Block 2
↓
...
↓
Transformer Block N
↓
LM head / output layer
↓
Logits
↓
Softmax (conceptually)
↓
Next-token probabilities
  
```

During generation, the model selects or samples a next token, appends it to the context and repeats the process.

6. Tokenization

Neural networks operate on numerical representations, not raw text. A tokenizer converts text into tokens and then token IDs.

```

"I love machine learning"
↓
["I", "love", "machine", "learning"]
↓
[ID1, ID2, ID3, ID4]
  
```

- A token is not necessarily a whole word. It can be a word, subword, punctuation mark or other learned unit.
- The tokenizer and model vocabulary must be compatible.
- Tokenization affects sequence length, memory and cost.

7. Embeddings

Each token ID is mapped to a dense vector. The vector is the representation that enters the neural network.

```
Token ID
↓
Embedding lookup
↓
[0.12, -0.44, 0.81, ...]
```

If the hidden size is `d_model`, each token is represented by a vector of approximately `d_model` dimensions. A sequence becomes a matrix `X` with shape roughly `[sequence_length, hidden_size]`.

8. Positional information

Attention itself does not inherently encode the order of tokens. The model therefore needs a mechanism to represent position.

- Older Transformer descriptions often use sinusoidal positional encodings.
- Some models use learned positional embeddings.
- Many modern LLMs use Rotary Position Embeddings (RoPE) or related mechanisms.

Key point: Do not claim every Transformer uses the exact same positional mechanism. It is architecture-dependent.

9. The Transformer block

```
Input
↓
Normalization
↓
Self-Attention
↓
Residual connection
↓
Normalization
↓
MLP / Feed-Forward network
↓
Residual connection
↓
Output
```

Exact ordering and normalization style vary between architectures. Modern decoder-only LLMs commonly use pre-normalization variants and other refinements.

- **Self-attention:** mixes information across tokens.
- **MLP:** performs learned nonlinear transformations independently at each token position after attention.
- **Residual connections:** help information and gradients flow through deep networks.
- **Normalization:** stabilizes activations and training.

10. Self-Attention — the core idea

For every token, the model asks: *which other tokens should influence my representation, and by how much?*

```
Sentence:
"The animal didn't cross the street because it was tired."
```

```
For "it", attention may assign strong weight to
relevant earlier tokens such as "animal" while
```

assigning different weights to other tokens.

Key point: Attention is a learned weighted information-mixing mechanism. It is not a database lookup or a human-readable reasoning table.

11. Query, Key and Value

Given input matrix X , learned projection matrices produce:

```
Q = X W_Q
K = X W_K
V = X W_V
```

- **Query (Q):** what information this token is looking for.
- **Key (K):** what kind of information each token offers for matching.
- **Value (V):** the information that is actually aggregated.

A useful analogy is a search system: a query is matched against keys, and the corresponding values are weighted and combined.

12. Scaled Dot-Product Attention

The standard formula is:

```
Attention(Q, K, V)
= softmax( Q K^T / √d_k ) V
```

Step by step:

1. Compute QK^T : similarity/compatibility scores between queries and keys.
2. Divide by $\sqrt{d_k}$: scale the dot products to keep their magnitude controlled.
3. Apply softmax: convert scores into normalized attention weights.
4. Multiply by V : produce a weighted combination of value vectors.

Key point: The output for each token contains information gathered from other tokens according to learned attention weights.

13. Why divide by $\sqrt{d_k}$?

If the key/query dimension is large, dot products can have large variance. Very large logits can make softmax extremely peaked and can make optimization less stable. Scaling by $\sqrt{d_k}$ helps keep the scores in a useful range.

```
score = QK^T
scaled_score = score / √d_k
weights = softmax(scaled_score)
```

14. Softmax

Softmax turns a vector of scores into positive values that sum to 1.

```
scores = [2.0, 1.0, 0.1]
softmax(scores) ≈ [0.66, 0.24, 0.10]
```

These values can be interpreted as relative attention weights. The model then uses them to weight the corresponding V vectors.

15. Self-attention vs cross-attention

- **Self-attention:** Q, K and V come from the same sequence/representation.
- **Cross-attention:** queries come from one representation while keys and values come from another.

Encoder-decoder Transformers commonly use cross-attention in the decoder to attend to encoder outputs. Pure decoder-only LLMs generally rely on self-attention within the language model blocks.

16. Multi-Head Attention

Instead of one attention operation, Transformers can use multiple attention heads.

Input
↓
Head 1 ■■
Head 2 ■■
Head 3 ■■ → concatenate → output projection

Each head has its own learned projections and can focus on different patterns or relationships. The exact learned role of a head is not fixed in advance.

- More heads provide multiple attention subspaces.
- Outputs are concatenated and projected back to the model dimension.

17. Causal attention in GPT/Llama-style LLMs

Autoregressive decoder-only language models predict the next token using previous context. They must not use future tokens when predicting a current position.

Tokens: I love machine learning

```
Attention:
I  ✓ ✗ ✗ ✗
love ✓ ✓ ✗ ✗
machine ✓ ✓ ✓ ✗
learning ✓ ✓ ✓ ✓
```

This is implemented with a causal mask, often represented as a lower-triangular attention pattern.

Key point: Causal masking is why next-token prediction can be trained without letting the answer leak into the input context.

18. Feed-Forward Network / MLP

After attention, a Transformer block applies a learned nonlinear transformation to each position.

Simplified:

$$\text{FFN}(x) = W \sigma(Wx + b) + b$$

Modern LLMs frequently use gated MLP variants rather than this exact simple two-layer form. In model implementations you may see modules such as:

gate_proj
up_proj
down_proj

Key point: These are associated with the MLP portion, not the attention mechanism.

19. Residual connections

[illegible]



Atte

↓ ■

Tran

↓ ■

+ ☐ ☐

Output

Residual connections add the block input back to the transformed output. This helps deep networks preserve information and improves gradient flow.

20. Normalization

Normalization layers help control the distribution and scale of activations. Modern LLMs commonly use LayerNorm or RMSNorm variants.

- The exact normalization choice depends on the architecture.
- Normalization is one of several techniques that make deep Transformer training more stable.

21. How a Transformer becomes an LLM

A Transformer architecture becomes a language model when it is trained on language data with a language-modeling objective. In an autoregressive LLM, the common objective is next-token prediction.

"The sky is"

"blue"

negative and positive and negative sample populations

Pretraining repeats this over huge amounts of text, allowing the model to learn syntax, semantics, patterns and many useful statistical relationships.

22. Logits, probabilities and next-token prediction

The final hidden representation is mapped to vocabulary-sized logits.

Se

The model does not directly output a sentence as one indivisible object. Generation is iterative: one token is produced, then it becomes part of the context for the next prediction.

23. Training vs inference

Stage	What happens
Training	Tokens are processed, loss is calculated against targets, gradients are computed and parameters are updated.

Stage	What happens
Inference	The model predicts tokens using learned parameters; gradients and weight updates are normally disabled.

24. Encoder, decoder and encoder-decoder

Type	Main idea	Examples / use
Encoder-only	Build contextual representations of input	BERT; classification, extraction
Decoder-only	Autoregressive next-token generation	GPT, Llama, Qwen; chat/generation
Encoder-decoder	Encode input then generate output using decoder	T5; translation/summarization

Key point: GPT/Llama-style systems are generally decoder-only Transformer language models. BERT is an encoder-based Transformer model.

25. The exact connection to Llama and QLoRA

This is where Transformer knowledge becomes useful for the fine-tuning concepts you have been studying.

```

Llama
↓
decoder-only Transformer LLM
↓
many Transformer blocks
↓
each block contains attention + MLP
↓
attention has projections such as q_proj/k_proj/v_proj/o_proj
↓
MLP may have gate_proj/up_proj/down_proj

```

When LoRA is applied to these modules, it adds trainable low-rank updates to selected linear transformations while the pretrained base weights are typically frozen.

26. LoRA connection

Suppose a pretrained linear layer has weight matrix W . Full fine-tuning updates W directly. LoRA keeps W frozen and learns a low-rank update.

$$W' = W + \Delta W$$

LoRA:

$$\Delta W = B A$$

where A and B are much smaller matrices than W .

The rank r is intentionally small. This reduces the number of trainable parameters.

- $q_proj/k_proj/v_proj/o_proj \rightarrow$ attention-side linear projections.
- $gate_proj/up_proj/down_proj \rightarrow$ MLP-side linear projections.

Key point: The list of target modules is a configuration choice; not every LoRA run must target every listed module.

27. Quantization and QLoRA connection

Quantization represents model weights using fewer bits. QLoRA combines a quantized frozen base model with trainable LoRA adapters.

```

Pretrained Transformer LLM
↓
4-bit quantized base weights

```

```

↓
base model remains frozen
+
LoRA adapters remain trainable
↓
task-specific dataset
↓
update adapters

```

A common QLoRA setup uses a 4-bit NormalFloat (NF4) representation for the base model and additional memory-saving techniques. Exact implementation details depend on the software stack and model.

28. Why Transformer understanding matters for fine-tuning

- You need to know where attention lives so you can understand `q_proj`, `k_proj`, `v_proj` and `o_proj`.
- You need to know the MLP to understand `gate_proj`, `up_proj` and `down_proj`.
- You need to know model parameters to understand what is frozen and what LoRA trains.
- You need to know causal language modeling to understand the training target and loss.
- You need to know sequence length because attention and memory requirements depend heavily on it.

29. Complexity — why long context is expensive

For standard full self-attention over sequence length n , the attention score matrix has roughly $n \times n$ entries. This leads to $O(n^2)$ attention-score computation and memory in the usual formulation.

`n = sequence length`

Attention score matrix:
`[n × n]`

`n` doubles → number of pairwise attention entries
 can grow by about 4×

This is one reason long-context training and inference can be expensive. Modern systems use optimizations and alternative attention mechanisms to improve practical efficiency.

30. KV cache during generation

During autoregressive generation, the model repeatedly attends to previous tokens. A key/value cache stores previously computed K and V tensors so they do not need to be recomputed from scratch at every generation step.

```

Prompt → compute K/V
↓
store in KV cache
↓
next token → reuse previous K/V + compute new token
↓
repeat

```

Key point: KV cache is an inference optimization. It is not the same thing as model weights or training memory.

31. Attention intuition with a concrete sentence

Sentence: **'Riya put the book on the table because it was heavy.'**

When processing 'it', the model can use attention to combine information from earlier tokens. The attention pattern is learned from data; it is not a hard-coded rule saying 'it always means the nearest noun'.

```
"it"
↓
attention scores over previous tokens
↓
softmax weights
↓
weighted sum of V
↓
contextual representation for "it"
```

32. A complete mental model

TRANSFORMER-BASED LLM

```

Text
↓
Tokenizer
↓
Token IDs
↓
Embeddings + position
↓
████████████████████████████████████████████████████████████████████████████████
■ Transformer Block ■
■ ■
■ Self-Attention ■
■  $X \rightarrow Q, K, V$  ■
■  $QK^T / \sqrt{d} \rightarrow \text{softmax} \rightarrow \text{weighted } V$  ■
■ ↓ ■
■ Residual + normalization ■
■ ↓ ■
■ MLP / Feed-Forward ■
■ gate / up / down (architecture ■
■ dependent) ■
■ ↓ ■
■ Residual + normalization ■
████████████████████████████████████████████████████████████████████████████████
↓
repeat N times
↓
LM Head
↓
Logits
↓
Next-token probability
↓
Next token

```

33. What happens during pretraining?

```
graph TD
    A[Large text corpus] --> B[tokenization]
    B --> C[input/target token sequences]
    C --> D[Transformer forward pass]
    D --> E[next-token logits]
    E --> F[cross-entropy loss]
    F --> G[backpropagation]
    G --> H[optimizer updates model parameters]
```

↓
repeat many times

The resulting model learns general language patterns. Pretraining is usually far more computationally expensive than task-specific fine-tuning.

34. What happens during supervised fine-tuning?

Pretrained LLM
↓
instruction/task dataset
↓
forward pass
↓
loss on training targets
↓
backpropagation
↓
update selected parameters
↓
task/domain-adapted model

With LoRA/QLoRA, the pretrained base weights are usually frozen and the small adapter parameters are updated.

35. RAG vs Transformer vs fine-tuning

Concept	What it solves
Transformer	Architecture used to process/transform representations and, in decoder models, generate tokens.
RAG	Supplies external retrieved information at inference time; it does not require changing model weights.
Fine-tuning	Changes model parameters/adapters so behavior or task performance is adapted.

In a real application, these can be combined: a Transformer-based LLM can use RAG for fresh/private knowledge and LoRA/QLoRA fine-tuning for behavior, formatting or domain adaptation.

36. Common misconceptions

- 'Transformer means ChatGPT.' → Transformer is an architecture; ChatGPT is a product/system built using models and additional components.
- 'Every Transformer is an LLM.' → No. Transformers are used in vision, speech and many non-LLM systems.
- 'Dataset is fine-tuned.' → The model or its trainable adapters are trained using the dataset.
- 'LoRA trains the whole LLM.' → Standard LoRA freezes the base model and trains small low-rank adapters.
- 'QLoRA means a completely different model architecture.' → QLoRA is a parameter-efficient fine-tuning approach combining quantization with LoRA.
- 'Attention is just a lookup table.' → Attention computes learned, input-dependent weighted combinations of representations.

37. Interview questions — basic

Q1. What is a Transformer?

A Transformer is a neural-network architecture based on attention mechanisms. It processes token representations and learns relationships between them. Modern language models such as GPT and Llama are commonly based on decoder-only Transformer architectures.

Q2. Why are Transformers better suited to parallel training than RNNs?

Transformers can process token positions in parallel during training because self-attention does not require recurrent hidden-state updates one token at a time. Autoregressive generation is still sequential, but training can be highly parallelized.

Q3. What is self-attention?

Self-attention computes how much each token should attend to other tokens in the same sequence and produces a context-aware representation using weighted combinations of value vectors.

Q4. What are Q, K and V?

Q is the query representation, K is the key representation used for matching, and V is the information aggregated according to the resulting attention weights.

Q5. Give the attention formula.

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$$

Q6. Why use multiple attention heads?

Multiple heads allow the model to learn different attention patterns in different learned subspaces and then combine them.

Q7. What is causal masking?

A causal mask prevents a token position from attending to future positions, enabling autoregressive next-token prediction without future-token leakage.

Q8. What is the role of the MLP?

The MLP applies a learned nonlinear transformation to each token representation after attention. Modern LLMs often use gated MLP variants.

38. Interview questions — intermediate

Q9. Difference between encoder-only and decoder-only Transformers?

Encoder-only models build bidirectional contextual representations and are useful for understanding tasks. Decoder-only models use causal attention and are designed for autoregressive generation. BERT is encoder-based; GPT/Llama are decoder-only examples.

Q10. Why does a Transformer need positional information?

Because attention by itself does not inherently tell the model the order of tokens. Positional mechanisms encode token order or relative position information.

Q11. What is the difference between self-attention and cross-attention?

In self-attention, Q, K and V originate from the same sequence. In cross-attention, Q comes from one representation while K and V come from another, allowing one sequence to attend to another.

Q12. What are residual connections?

They add a block's input to its transformed output, helping optimization and information/gradient flow in deep networks.

Q13. What is the LM head?

It maps the final hidden representation into vocabulary-sized logits so the model can assign scores/probabilities to possible next tokens.

39. Interview questions — fine-tuning connection

Q14. How does LoRA modify a Transformer?

LoRA freezes the pretrained weight matrix W and learns a low-rank update $\Delta W = BA$. It is commonly applied to selected linear projections in attention and/or MLP modules.

Q15. What are q_proj , k_proj and v_proj ?

They are learned linear projections that create Q, K and V representations for attention. o_proj is the output projection after attention heads are combined.

Q16. What are $gate_proj$, up_proj and $down_proj$?

They are common names for projections in gated MLP/feed-forward blocks of modern decoder LLMs. Exact naming and structure vary by architecture.

Q17. Why use QLoRA?

QLoRA reduces memory requirements by keeping the pretrained base model quantized while training LoRA adapters. This can make fine-tuning large models feasible on more limited hardware.

Q18. Where does Unsloth fit?

Unsloth is an optimization/toolkit layer used to make LLM training and inference workflows more efficient. It is not itself the Transformer architecture or a replacement for LoRA/QLoRA.

40. Interview questions — advanced

Q19. What is the computational issue with standard self-attention?

The attention score matrix scales quadratically with sequence length, giving $O(n^2)$ pairwise attention computation/memory in the standard formulation.

Q20. What is KV caching?

KV caching stores previously computed key/value tensors during autoregressive generation so they can be reused, avoiding repeated computation for all previous tokens at each step.

Q21. Why is attention divided by $\sqrt{d_k}$?

To control the scale of dot products as the key/query dimension grows, preventing excessively large softmax inputs and helping stable optimization.

Q22. Is attention the same as human reasoning?

No. Attention is a differentiable weighted aggregation mechanism. It can contribute to learned representations and useful reasoning behavior, but an attention weight should not automatically be interpreted as a complete explanation of human-like reasoning.

41. 60-second interview answer

“A Transformer is a neural-network architecture centered around self-attention. Text is tokenized and converted into embeddings with positional information, then passed through multiple Transformer blocks. Each block typically contains attention and an MLP with residual connections and normalization. In self-attention, the model creates

Query, Key and Value projections and computes scaled dot-product attention: $\text{softmax}(QK^T/\sqrt{d_k})V$. Decoder-only Transformers use causal masking for autoregressive next-token prediction. Modern LLMs such as Llama and GPT are based on this architecture. For fine-tuning, methods like LoRA and QLoRA modify selected Transformer layers efficiently instead of necessarily updating every pretrained parameter.”

42. The hierarchy you should memorize

```
Architecture
↓
Transformer

Model
↓
Llama / GPT / Qwen

Language-model objective
↓
Next-token prediction

Fine-tuning
↓
SFT / PEFT

PEFT method
↓
LoRA

Quantized PEFT
↓
QLoRA

Quantization format / technique
↓
4-bit / NF4 (common QLoRA setup)

Efficiency toolkit
↓
Unsloth
```

43. Final revision sheet

- Transformer = architecture.
- Self-attention = tokens exchange information using learned weights.
- Q = query, K = key, V = value.
- Attention = $\text{softmax}(QK^T/\sqrt{d_k})V$.
- Multi-head attention = multiple attention subspaces combined.
- Causal mask = no future-token access in autoregressive decoding.
- MLP = nonlinear transformation inside each block.
- Residual + normalization = important deep-network components.
- Decoder-only Transformer = common architecture for GPT/Llama/Qwen-style LLMs.
- LLM = large language model, often Transformer-based.
- LoRA = low-rank trainable update to selected weights.
- QLoRA = quantized frozen base + LoRA adapters.
- Unsloth = optimization/toolkit for efficient LLM workflows.

- RAG = retrieves external information; it is not the Transformer itself.
- Fine-tuning = train model parameters or adapters using task-specific data.

44. Practical project architecture

```
User
↓
Application / API
↓
(Optional) RAG retrieval
↓
Prompt construction
↓
Transformer-based LLM
↓
Tokenizer / model inference
↓
Generated response
```

```
If adapting the model:
Dataset
↓
SFT
↓
LoRA / QLoRA
↓
Transformer weights frozen + adapters trained
↓
Save adapter / model
↓
Inference
```

This is the mental model to carry into your projects: the Transformer is the neural architecture inside the model; training/fine-tuning changes learned parameters; RAG supplies external context; and libraries such as Hugging Face or Unsloth provide implementation and optimization tools.